

Node.js Recap

Javascript as a programming language

Standalone pages used JS for dynamic content and updates

jQuery / AJAX

Created a library for Frontend Development - React

Created a runtime environment outside of the Browser

- pulled V8 engine from the Chrome browser
- redesigned it to work independently on the system resources
- used this V8 to create an environment to execute Javascript
- It also granted access to system resources like I/O and file system
- Named it as Node.js

JS is single threaded and async in nature, it works very fast and can switch the tasks as soon as the processor available.

How Node.js works?

- uses V8 engine to convert JS code to machine code
- libuv library to handle async operations
- used event loop to execute async operations

Setup of Node Application:

1. npm init - initialize the folder as a node project
2. define the metadata as prompted in the CLI.
3. Once metadata is added folder is created with initial details in package.json

When a package is installed:

1. the package itself is added to node_modules
2. its dependencies are installed to node_modules
3. dependencies of the package dependencies are also added to node_modules

Types of Packages

1. Built-in Packages - provided by the node.js itself
2. Third Party - Installed from the npm registry
3. Custom - Modules / Packages that we create

HTTP and Express

HTTP is built in Node.js package for server creation

Express is the third party package for server creation

- create web servers
- handle HTTP requests (GET, POST, PUT, DELETE etc.)
- send responses
- directly work with the HTTP protocol

http.createServer -> creates a server

req -> request object (received from the client)

res -> response object (to be sent to the client)
res.writeHead -> set headers and status code
res.end -> it send the response and end the request
server.listen -> starting the server on the mentioned port number

Express

It is a lightweight web application framework built on Nodejs HTTP module

- Server creation
- Routing
- Middleware handling
- Request and Response management

npm i express

import express from 'express'

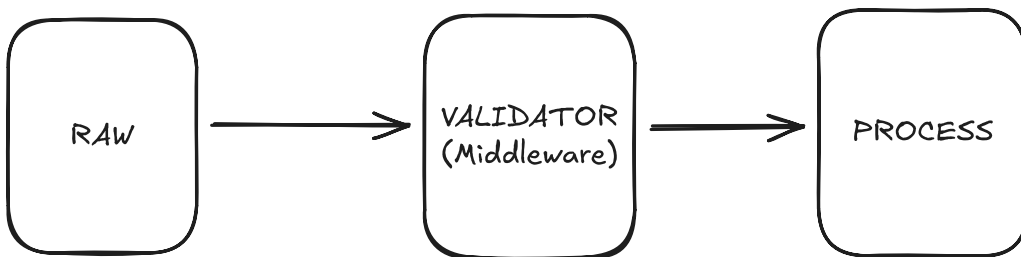
- defining a dependency on express and importing it from node_modules

express()

- it creates an application object
- it setup internally:
 - routing
 - middleware stack
 - HTTP server

const app = express()

app is the function and object that handle the HTTP requests



HTTP vs Express

HTTP

- full control over the things
- we can write more code
- low level systems

Express

- high level systems
- less code required
- faster development

```
app.listen(PORT_NUMBER, callback())
```

- it creates a HTTP server
- binds server to the mentioned port
- starts listening for the request
- callback is executed only once server is ready

Ports

80 - HTTP Request

443 - HTTPS Request

3306 - MySQL

27017 - MongoDB

3000 - Development servers

8000 - Backend servers

```
app.get('/', (req,res) => {})
```

- defines a GET route with path as '/'
- callback executed when route matches
- req contains the request object from the client
- res contains the response object to be sent to the client

What are Middleware?

Middleware are functions that execute between the request and the response in the application

- execute before the route handlers
- can modify the request and the response
- can also stop the current request cycle

Client -> Middleware1 -> Middleware2 -> Route -> Response

/dashboard/1

- auth middleware - check if user has valid credentials
- re-route to login page

Why middleware is used?

- authenticating user
- log requests
- parse the JSON body
- enable CORS
- modify request or response
- error handling

```
app.use((req, res, next) => {  
  // do something  
  next()  
})
```

next will move the execution to the route handler

if next is not present

- no response generated
- request keeps ongoing forever

Types of Middlewares

1. Inbuilt

- `json()` - to parse the JSON object from the request body
- `urlencoded()` - to parse the form-data from the request
- `static(PATH)` - to serve the files

2. Third Party

- `cors` -> allow the cross-origin requests

CORS - Cross Origin Resource Sharing

- configurations for CORS allows which website can access the data

Frontend URL - `http://localhost:3000`

API URL - `http://localhost:8000`

if CORS is not available Frontend will not be able to call the APIs

origin is different if:

1. protocol is not the same
 - `http://example.com` as my registered host
 - `https://example.com` as my request source
2. Domain difference
3. Port Different

Order of middleware execution is from top to bottom

3. Custom Middleware

- create middleware as per our requirement